Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Bu ders, uygulama saatleriyle birleştirilmiş uzun bir derstir. Fatih Hoca'nın ders notlarındaki sorular çözülür: iki değişkenin yer değiştirmesi (swap), toplamı 1000 olan Pisagor üçlüsü, Floyd üçgeni, dizinin ters çevrilmesi, palindrom kontrolü ve palindrom sayılar. Her soruda önce basit çözüm kurulur, sonra adım sayısı azaltılır. Dersin sonunda laboratuvar sınavının kuralları anlatılır.
- Ofis saatleri
  - Salı ve çarşamba 12.00-13.00. Bu saatler dışında görüşmek için Yıldız veya Gmail adresinden en az 3-4 gün önce randevu istenmelidir.
- Swap (iki değişkenin yer değiştirmesi)
  - Swap, sıralama algoritmalarında ve araya eleman eklerken kaydırma gibi işlemlerde kullanılır.
  - Bazı işlemcilerde iki register'ın veya bellek hücresinin içeriğini tek komutla değiştiren bir `XCHG` (exchange) komutu vardır. C'de böyle bir komut yoktur.
  - Geçici değişkenle (temp): `tmp = x; x = y; y = tmp;`. `tmp` dışta kalır, `x` ve `y` çapraz olarak birbirine atanır. Ödünleşim: 1 ek bellek, 3 atama.
  - Atama (`=`) değeri kopyalar, kaynağı boşaltmaz.
  - Üç değişkende döndürme (`x`'i `y`'ye, `y`'yi `z`'ye, `z`'yi `x`'e): `tmp = z; z = y; y = x; x = tmp;`.
  - Bu yedekleme mantığı Fibonacci'de ve "en büyük iki not" probleminde de kullanılmıştı.
- Geçici değişkensiz swap yöntemleri
  - Toplama-çıkarma: `x = x + y; y = x - y; x = x - y;`.
  - Çarpma-bölme: `x = x * y; y = x / y; x = x / y;`.
  - XOR: `x = x ^ y; y = x ^ y; x = x ^ y;`. C'de `^` üs değil, bit düzeyinde XOR operatörüdür. XOR iki bit aynıysa 0, farklıysa 1 verir. Kriptolojide mesaj ile anahtarın XOR'lanmasıyla şifreleme yapılır.
  - Üç yöntemde de ek bellek yoktur, ama 3 atamaya ek olarak 3 aritmetik veya mantıksal işlem yapılır. Bu yine bir trade-off'tur. Seçim probleme ve donanıma göre yapılır.
  - Geçici değişkenle tip uyumsuzluğu sorunu olmaz. Toplama ve çarpmada ara sonuç tipin sınırını aşabilir (ör. iki tane 2 milyarlık `int`'in toplamı). Bölmede sıfıra bölme riski vardır. Bunlar doğruluğu (correctness) bozar.
- Toplamı 1000 olan Pisagor üçlüsü ($a + b + c = 1000$, $a^2 + b^2 = c^2$)
  - 1. yöntem: `a`, `b`, `c` için 1'den 999'a üç iç içe döngü ve iki şartın kontrolü. Yaklaşık 10^9 tur.
  - Bir öğrenci $c^2 - b^2 = a^2$'yi $(1000 - a)(c - b) = a^2$ olarak düzenledi. Bu eşitlik $a + b + c = 1000$ kullanılarak türetildiği için, `c` ayrı bir döngüyle serbestçe seçilince toplam şartı ayrıca kontrol edilmelidir. Denemede toplamı 1000 olmayan çok sayıda sonuç da çıktı. Hoca bunun için "600 ve 895" dedi, ama bu iki değer formülün aynı üçlüsünde birlikte geçmiyor (ders sonrası denemede 2321 üçlü, yalnız ikisinin toplamı 1000). Hoca büyük olasılıkla ekrandaki farklı satırları okudu.
  - 2. yöntem: `c` için döngüye gerek yoktur: `c = 1000 - a - b`. Ayrıca `b`, `1000 - a`'dan küçük olmalıdır, yoksa `c` sıfır veya negatif olur. İki döngü kalır: `a` 1'den 998'e, `b` 1'den `1000 - a - 1`'e.
  - 3. yöntem: öğrenci önerileriyle `a` 500'ün altına sınırlandı (c hem `a`'dan hem `b`'den büyük olmak zorunda olduğu için). 4. yöntem: `a` yine `a < 999` ile döner, `c`'nin `a` ve

b'den büyük olma şartı bir `while` / `do-while` ile kontrol edilir.
  - Sayaçlarla ölçülen tur sayıları: 997.002.999, 498.501, 124.750 ve 167.165. Bütün yöntemler aynı sonucu buldu: 200, 375, 425 (ve sırası değişmiş 375, 200, 425).
  - Burada `while` yerine `for` kullanılır, çünkü ilk bulunan çözümde durulmaz, bütün çözümler aranır.
  - `&&` ile bağlanan şartları paranteze almak "garantici" bir yaklaşımdır. Operatör önceliği her zaman akılda olmayabilir.
- Floyd üçgeni
  - Satırlarda 1; 2 3; 4 5 6; 7 8 9 10; ... sayıları vardır. Soru: n. satırın son elemanı kaçtır?
  - 1. çözüm: iç içe iki döngü, `i` 1'den n'e, `j` 1'den `i`'ye, her turda `t = t + 1`. Tur sayısı $\frac{n(n+1)}{2}$.
  - 2. çözüm: n. satırın son elemanı, bir önceki satırın son elemanına n eklenerek bulunur. `t = 1` ile başlanır, `i` 2'den n'e `t = t + i`. Tur sayısı $n - 1$.
  - 3. çözüm: son eleman $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$'dir. 3 işlemde bulunur.
  - Adım sayısı n^2 mertebesinden n'e, oradan sabite indi. Algoritma geliştirmenin iteratif bir süreç olduğu böyle görülür.
- Dizinin ters çevrilmesi
  - 8 elemanlı dizide 1 ile 8, 2 ile 7, 3 ile 6, 4 ile 5 yer değiştirir. 7 elemanlı dizide ortadaki eleman (4) yerinde kalır.
  - Döngü dizinin yarısına kadar döner: `for (i = 0; i < n/2; i++)`, gövdede `dizi[i]` ile `dizi[n-i-1]` swap edilir.
  - Döngü sonuna kadar dönerse her eleman iki kez yer değiştirir ve dizi eski hâline döner.
  - `n/2` tam sayı bölmesidir. Tek sayıda elemanda aşağı yuvarlanır ($\lfloor n/2 \rfloor$), böylece ortadaki eleman yer değiştirmez. Akış diyagramında bu $\lfloor\rfloor$ ile gösterilmelidir.
  - Akış diyagramında indisler 1'den başlıyorsa karşı eleman `A[n - i + 1]`'dir. C'de indisler 0'dan başladığı için `A[n - i - 1]` olur.
  - Bir öğrenci swap'tan önce iki elemanın eşit olup olmadığını kontrol etmeyi önerdi. Hoca, bu kontrolün çoğu durumda gereksiz işlem ekleyeceğini söyler.
  - Eleman sayısı bilinmediği için dizinin boyutu `#define MAX 100` ile büyük tanımlanır, kullanıcının girdiği `n` kadarı kullanılır.
  - Dizinin sınırı dışına erişim (ör. `i <= n`) hata vermeyebilir. Okunan değer 0 da olabilir, büyük bir "çöp" değer de. C belleği sıfırlamaz. İşletim sistemi başka bir prosesin bellek alanına erişime izin vermez.
- Palindrom kontrolü
  - Palindrom, tersi kendisine eşit olan dizi veya string'dir (ör. "abba", 1441).
  - Naif çözüm: diziyi ikinci bir diziye ters kopyala (`B[i] = A[n-i-1]`), sonra iki diziyi baştan karşılaştır. Fark bulunur bulunmaz durmak için karşılaştırma `while` ile yapılır. Hoca, bu çözümün birçok kitapta olduğunu ama bilgisayar mühendisinden beklenmediğini söyler, çünkü ek bir dizi gerektirir.
  - Daha iyi çözüm: ek dizi kullanmadan `A[i]` ile `A[n-i-1]` karşılaştırılır. `while (A[i] == A[n-i-1] && i < n/2) i++;`. Döngüden sonra `i < n/2` ise palindrom değildir, değilse palindromdur.
- 6 basamaklı palindrom sayılar
  - Naif çözüm: 100.000'den 999.999'a her sayının 6 basamağı bölme ve mod ile ayrılır ve 1. ile 6., 2. ile 5., 3. ile 4. basamaklar karşılaştırılır. 900.000 sayı için en az 6'şar işlem yapılır.
  - Karşılaştırmalar iç içe `if` ile yazılmalıdır: ilk eşitlik sağlanmazsa diğerlerine bakılmaz. Bağımsız `if`'ler hem gereksiz iş yapar hem de sayacı yanlış artırabilir. Bir öğrenci basamakları da yalnızca gerektiğinde hesaplamayı önerdi. İşlem sayısı yaklaşık 6 × 900.000'den 2 × 900.000'e iner.
  - Üretme yaklaşımı (bir öğrencinin önerisi): sayı ABCCBA biçimindedir. `A` 1'den 9'a, `B` ve `C` 0'dan 9'a üç iç içe döngüyle sayı doğrudan üretilir: `A*100000 + B*10000 + C*1000 + C*100 + B*10 + A`. Tam 9 × 10 × 10 = 900 tur. Başka bir öğrencinin önerisiyle `A*100001 + B*10010 + C*1100` yazılarak çarpma sayısı azaltılır.
  - 7 basamaklı (ABCDCBA) için dört döngü gerekir. R basamak için gereken döngü sayısı R'ye bağlıdır ve iç içe döngü sayısı program yazılırken değişken yapılamaz.
- R basamaklı palindrom sayısı
  - 1 ve 2 basamakta 9, 3 ve 4 basamakta 90, 5 ve 6 basamakta 900 palindrom vardır.

Dıştaki basamak çifti 9 farklı (1-9), içteki her basamak çifti 10 farklı değer alır.
  - Genel formül: $9 \times 10^{\lfloor (R-1)/2 \rfloor}$.
  - Bunu hesaplamak için 10^t hesaplanmalıdır. Döngüyle $t = \lfloor (R-1)/2 \rfloor$ çarpma gerekir. Hızlı üs alma ile $\log t$ adıma inilebilir.
- Laboratuvar sorularından
  - Tek sayıların çarpımı, çift sayıların toplamı (1-20): toplam 0'dan, çarpım 1'den başlatılır. BBG 1 seviyesinde bir soru.
  - Girilen sayıyı ters çevirme: `while (tmp > 0) { ters = ters * 10 + tmp % 10; tmp /= 10; }`. Sayının kendisi bozulmasın diye `tmp`'ye kopyalanır.
  - Armstrong sayıları (100-1000): genel çözümde her sayının basamakları bir `while` döngüsüyle `sayi % 10` ve `sayi /= 10` ile alınır. Bir öğrencinin önerdiği gibi üç döngüyle üretmek de olur, ama bu yalnızca bu aralık için geçerli, özel bir çözümdür.
  - Onluktan ikiliğe çevirme: bir sonraki derse bırakıldı.
- Kosinüs serisi (ders sonrası soru)
  - Bir öğrenci $\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots$ serisini iki ayrı fonksiyonla (üs ve faktöriyel) hesaplıyordu.
  - Hoca, e^x'teki gibi önceki terimin değerini kullanmayı gösterdi: `ustlu = ustlu * x * x;` ve `t = 2` ile başlayıp `faktoriyel = faktoriyel * (t - 1) * t; t = t + 2;`. Böylece $0!$'dan $2!$'ye, $2!$'den $4!$'e geçilir.
  - Öğrencinin hızlı üs fonksiyonundaki hata Dev-C++'ta `printf` ile ara değerler yazdırılarak arandı. 2 ve 5 ile çağrılan fonksiyon 32 yerine 25 verince önce son bölümün diziye yazılmadığından şüphelenildi. Asıl durum, parametrelerin önce üs sonra taban almasıydı: $5^2 = 25$ doğruydu. İki sürümün farklı sonuç vermesinin sebebi de ikinci döngüden önce `r`'nin sıfırlanmamasıydı (03:32:15). `r` sıfırlanınca iki sürüm aynı sonucu verdi.

Hocanın Özellikle Vurguladığı Kısımlar

- Swap'ta `tmp` her zaman dışta kalır, değişkenler çapraz atanır. Bunu karıştıranlar yeterince kod yazmamış olanlardır.
- Geçici değişkensiz swap yöntemlerinin sınırları vardır: taşma, sıfıra bölme. Doğruluk algoritmanın en önemli özelliklerinden biridir.
- C'de dizi indisleri 0'dan başlar. Döngüler istenen yerden başlayabilir, ama dizilerde indis 0'dır. Buna alışılmalıdır.
- Dizilerde döngü şartı `i < n` olmalıdır, `i <= n` dizinin dışına taşar.
- Ek bellek kullanan naif çözümler (diziyi kopyalayıp karşılaştırmak) bir bilgisayar mühendisinden beklenmez.
- Bağımlı şartlar iç içe `if` ile yazılmalıdır. Bağımsız `if`'ler gereksiz iş yapar ve yanlış sonuç verebilir.
- Özel bir aralığa göre yazılmış çözüm ile genel çözüm arasındaki fark görülmelidir. Genel çözüm daha değerlidir.
- Laboratuvar ve sınavlarda kod standartlarına ve sorudaki kısıtlara uyulmalıdır. "Bu bir BBG dersi değil."
- Dördüncü sınıfa gelip bu derste çözülen soruları hâlâ çözemeyen öğrenciler vardır. Kalite için taviz verilmez.
- Sorular mail yerine ders sonunda sorulmalıdır.

Kısa Tekrar Notları

- Swap: `tmp = x; x = y; y = tmp;`.
- Tempsiz: `x+=y; y=x-y; x-=y;` (taşma riski), `x*=y; y=x/y; x/=y;` (sıfır riski), `x^=y; y^=x; x^=y;`.
- Pisagor üçlüsü: `c = 1000 - a - b`, iki döngü. Cevap: 200, 375, 425.
- Floyd üçgeninde n. satırın sonu: $n(n+1)/2$.
- Ters çevirme: `i < n/2`, swap `dizi[i]` ile `dizi[n-i-1]`.
- Palindrom: `while (i < n/2 && A[i] == A[n-i-1]) i++;`, sonra `i < n/2` ise değil.
- 6 basamaklı palindromlar: ABCCBA üretimi, 900 tur.
- R basamaklı palindrom sayısı: $9 \times 10^{\lfloor (R-1)/2 \rfloor}$.
- Sayıyı ters çevir: `ters = ters*10 + tmp%10; tmp /= 10;`.

Detaylı Açıklamalar

XOR ile swap örneği

`x = 2 = 0010`, `y = 4 = 0100`:

```
x = x ^ y  →  0010 ^ 0100 = 0110
y = x ^ y  →  0110 ^ 0100 = 0010   (y artık 2)
x = x ^ y  →  0110 ^ 0010 = 0100   (x artık 4)
```

Swap yöntemlerinin karşılaştırması

Yöntem	Ek bellek	Atama	Ek işlem	Risk
Geçici değişken	1	3	yok	yok
Toplama-çıkarma	0	3	3 toplama/çıkarma	taşma
Çarpma-bölme	0	3	3 çarpma/bölme	taşma, sıfıra bölme
XOR	0	3	3 XOR	yalnızca tam sayılar

Pisagor üçlüsü, ikinci yöntem

```
for (a = 1; a < 999; a++)
    for (b = 1; b < 1000 - a; b++) {
        c = 1000 - a - b;
        if (a*a + b*b == c*c)
            printf("%d %d %d\n", a, b, c);
    }
```

Ders sonrasında bu kod `gcc -Wall -std=c89` ile derlendi. 498.501 turda 200 375 425 ve 375 200 425 sonuçlarını verdi. Derste gösterilen sayaçla aynıdır.

Dizi ters çevirme ve palindrom

```
for (i = 0; i < n/2; i++) {
    tmp = dizi[i];
    dizi[i] = dizi[n-i-1];
    dizi[n-i-1] = tmp;
}

i = 0;
while (i < n/2 && dizi[i] == dizi[n-i-1])
    i++;
if (i < n/2)
    printf("palindrom degil\n");
else
    printf("palindrom\n");
```

Ders sonrasında denendi: 1 2 3 4 5 dizisi 5 4 3 2 1 oldu. 1 2 3 2 1 palindrom, 1 2 3 1 palindrom değil çıktı. Akış diyagramındaki sırada (`A[i] == A[n-i-1] && i < n/2`) da `i` hiçbir zaman `n/2`'yi geçmediği için dizi dışına erişilmez. Ama sınır kontrolünü önce yazmak daha güvenli bir alışkanlıktır.

Ders sonrası notlar

- **"Unstable condition" ifadesi:** Hoca, tek sayıda elemanlı dizide ortadaki elemanın kendisiyle yer değiştirmesini "unstable condition" olarak adlandırdı ve sıralama algoritmalarındaki "stable" özelliğiyle ilişkilendirdi. Sıralama algoritmalarında kararlılık (stability), eşit değerli elemanların sıralamadan sonra da birbirlerine göre ilk sıralarını korumasıdır. Bir elemanın kendisiyle yer değiştirmesi yalnızca gereksiz bir işlemdir, kararlılıkla ilgisi yoktur.

- **`int dizi[n]`:** Bir öğrenci boyutu kullanıcıdan alınan `n` ile dizi tanımlamayı sordu. Hoca bunun C'de olmadığını söyledi ve büyük değerlerde "segmentation fault" aldı. Bu yazım değişken uzunluklu dizi (VLA) olarak C99'da vardır, derste kullanılan C89'da yoktur (GCC yine de uzantı olarak kabul eder). Büyük değerlerde alınan hata, yerel dizinin stack'e sığmamasından kaynaklanır. Bu derste `#define MAX` ile sabit boyutlu dizi kullanılmalıdır.
- **XOR swap'ın sınırı:** Derste XOR'un taşma gibi sorunları olmayabileceği söylendi. Doğrudur: XOR taşma yapmaz, sıfırla da sorun çıkmaz. İki değişkenin değeri eşit olsa bile doğru çalışır (5 ve 5 ile denendi). Sorun yalnızca iki ifadenin aynı bellek hücresini göstermesidir (ör. `dizi[i]` ile `dizi[j]`, `i == j` iken): ilk adımda değer sıfırlanır ve kaybolur. Ayrıca XOR yalnızca tam sayı tiplerinde kullanılabilir.
- **İşaretli tam sayı taşması:** Toplama-çıkarma yönteminde taşma C standardına göre tanımsız davranıştır (undefined behavior). Pratikte çoğu derleyicide değer "sarılır" ve swap yine doğru sonuçlanır, ama buna güvenilmemelidir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.